— PORTFOLIO · 2026

Sean Kakhkhorov

Full-Stack Developer (AI & DevOps)

I build web products end to end and keep them running: **five live products**, from a creator marketplace to an AI bookstore, a cited visa-information beta and AI document review.

2 yrs
10 months of professional web development

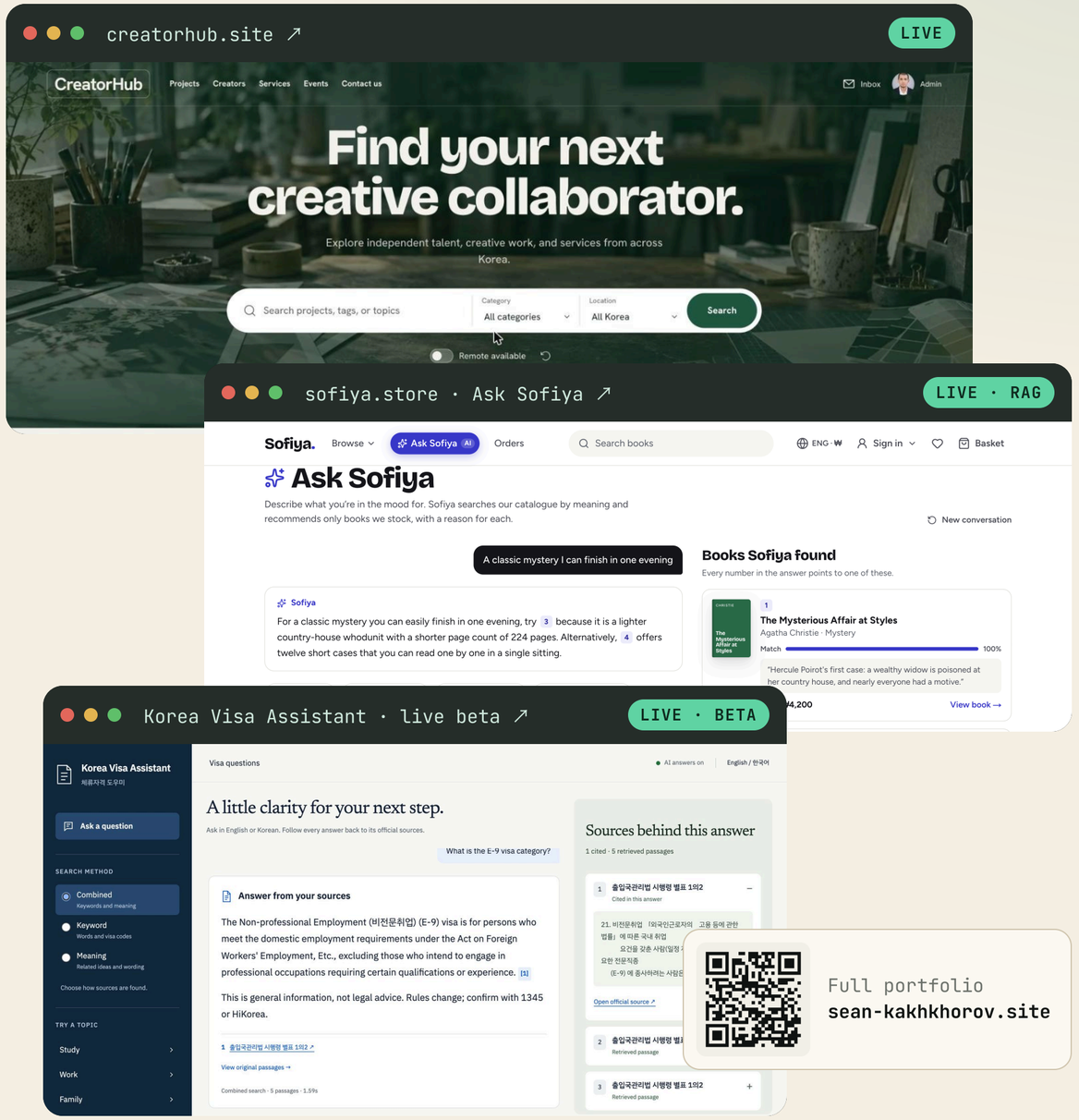
20+
client companies served at IT Local

5
live products: 4 solo, 1 collaborative

293
automated checks recorded in CreatorHub

CreatorHub ↗ · Sofiya ↗ · Korea Visa Assistant ↗ · CargoCheck AI ↗ · Lexara ↗ · sean-kakhkhorov.site

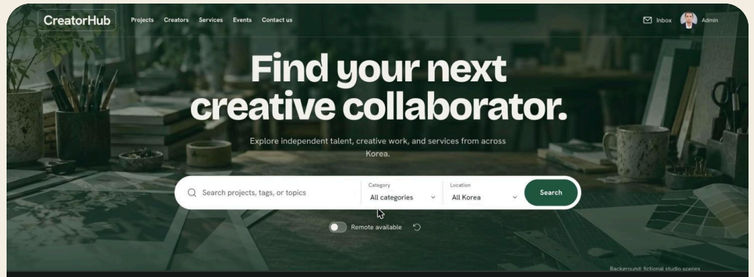
Suwon, South Korea · kakhkhorov97@gmail.com · github.com/kakhkhorov97



— PROJECTS AT A GLANCE

What I have built.

Also inside: About p. 03 · Experience p. 04 · Skills p. 05 · Contact p. 20

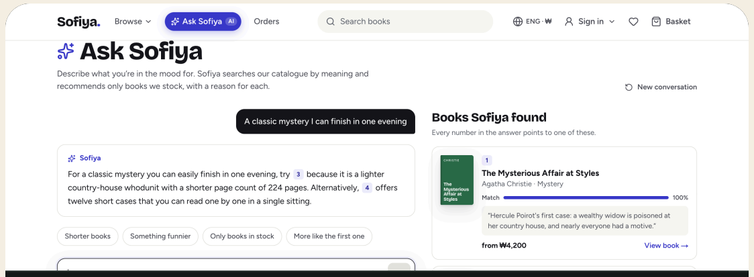


CreatorHub ↗ LIVE · SOLO

Creator marketplace for Korea with private chat, moderation and a Gemini assistant.

Next.js · NestJS · GraphQL · MongoDB

Open live app ↗ p. 06 →

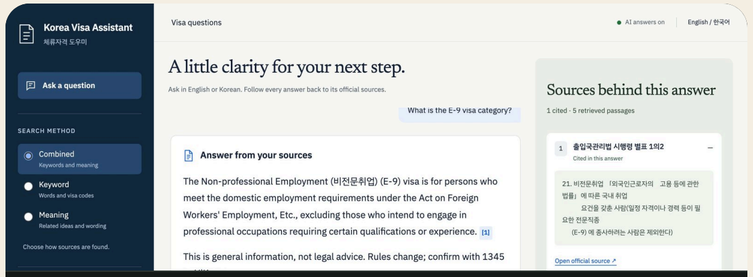


Sofiya ↗ LIVE · RAG

AI bookstore: a RAG concierge recommends only books in stock, with citations (hit@5 100%).

Next.js · NestJS · pgvector · Gemini

Open live site ↗ p. 10 →

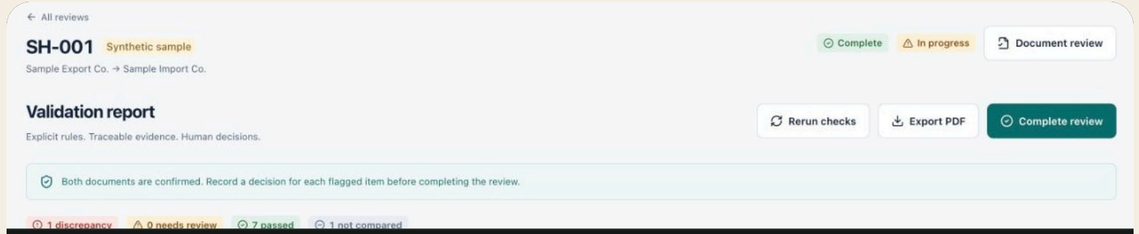


Korea Visa Assistant ↗ LIVE · BETA

Cited answers from selected official Korean visa sources, EN/KO; hybrid retrieval hit@5 0.969 on 32 questions.

Python · FastAPI · Chroma · Gemini

Open live beta ↗ p. 13 →

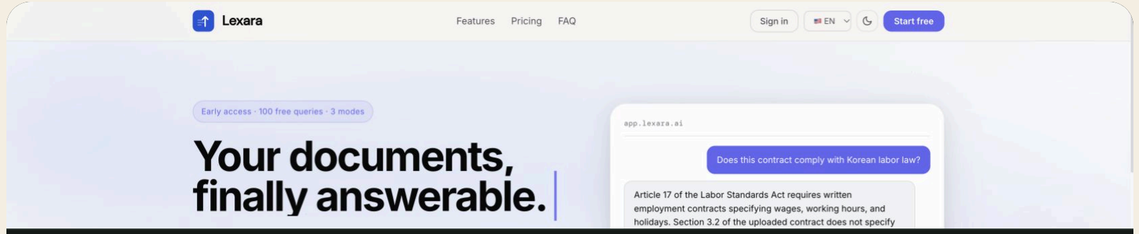


CargoCheck AI ↗ LIVE · GEMINI

AI document review: Gemini extraction, human confirmation, explicit rule checks.

React · NestJS · Neon · Upstash

Open live app ↗ p. 16 →



Lexara ↗ LIVE · COLLABORATIVE

Document Q&A with cited answers. I contributed as one of the developers.

FastAPI · PostgreSQL · React

Open public site ↗ p. 19 →

— ABOUT ME

Full-stack development, from code to production.

I started as a backend developer building Express and MongoDB APIs, then worked as a full-stack developer at IT Local from November 2023 to February 2026, in a team of four or more developers serving more than 20 client companies. There I looked after everything from database design and authentication to deployments, DNS and outages, which taught me to care about how code behaves after release, not just how it is written.

Since then I have shipped my own products: **CreatorHub ↗**, a creator marketplace I built and run alone; **Sofiya ↗**, an AI bookstore with a RAG book concierge; **Korea Visa Assistant ↗**, a public beta that explains Korean visa rules from official sources with citations; **CargoCheck AI ↗**, a live AI document-review app; and **Lexara ↗**, a document Q&A platform I helped another developer build.

I use coding agents such as Claude Code and Codex within written repository rules: I approve each change, and automated checks run before anything is merged.

Reliability
Monitoring, logs and quick outage fixes

Security
Auth, input validation and code review

Documentation
APIs and deploy steps written down

AI workflow
Coding agents with review gates



Kakhkhorov Shokhjakhon (Sean)
Full-Stack Developer (AI & DevOps)

— EXPERIENCE

Full-stack Web Developer
IT Local · 2023.11 – 2026.02

Backend Developer
Atlantis Investment · 2023.04 – 2023.09

— BASED IN

Suwon, Gyeonggi-do, South Korea

— LANGUAGES

Korean (business; 4-year degree taught in Korean)
English (fluent) · Uzbek (native) · Russian (elementary)

— FIND ME

github.com/kakhkhorov97
kakhkhorov97@gmail.com

— EXPERIENCE · IT LOCAL · ATLANTIS

Client websites, from build to servers.

IT Local builds and runs websites for other businesses. In a team of four or more developers I worked across the whole path, from the first page to the server it runs on. Client sites are not shown: they belong to IT Local's customers.

20+ client companies
Construction, retail, food service, corporate and public sector

→

Team of 4+ developers
Backend, frontend, servers and code review

→

CI deployment
Replaced manual deploy steps

→

5–15 Ubuntu servers
Nginx, PM2, Docker and Cloudflare

— THREE PROBLEMS I SOLVED

Manual deploys → CI

Problem. Releases were done by hand: SSH into each server, pull, install and restart. On 5–15 servers a missed step could break a site, and only a few people knew the steps.

What I did. Wrote the steps down, then moved them into a CI pipeline that installs, builds and restarts the service the same way every time.

Result. Every deploy followed the same reviewed steps, and other developers could release without me.

A client site goes down

Problem. Client sites occasionally went down or started returning errors behind Nginx.

What I did. Traced each incident through the Nginx, PM2 and application logs to the point of failure, fixed the underlying issue, and set PM2 to restart services automatically.

Result. The sites were brought back, and each fix was written down so the team could follow the same steps next time.

Slow pages as data grew

Problem. List and search pages got slower as client data grew.

What I did. Checked the slow queries, added indexes that match the filters and sort order, and paginated results instead of loading everything.

Result. Queries used the new indexes, so list and search pages stayed responsive as data kept growing. I did not record before-and-after timings.

Also at IT Local
Code review, API and deploy docs, React and Tailwind pages, a full handover.

Atlantis Investment · 2023.04 – 2023.09
Backend, alongside my studies: Express.js, MongoDB/Mongoose, JWT with bcryptjs, Multer uploads, search indexes.

— SKILLS & TOOLS

My stack.

CORE, USED DAILY ALSO USED

Frontend

React · Next.js · TypeScript · JavaScript · Apollo Client · Redux Toolkit · MUI · Tailwind CSS · SCSS · HTML · CSS · jQuery · Axios · TanStack Query · React Router · Socket.IO Client · Framer Motion · Three.js · Swiper · Anime.js · Typed.js · TUI Editor · Bootstrap · Chakra UI · shadcn/ui · Figma

Backend

Node.js · NestJS · Express · GraphQL · REST APIs · Socket.io · JWT & OAuth · Python · FastAPI · Zod · Role-based access · EJS · bcryptjs · Multer

Databases

PostgreSQL (Neon) · pgvector · Drizzle ORM · MongoDB · Mongoose · Redis (Upstash) · Transactions · Indexing & query tuning · BullMQ · Chroma · MySQL

DevOps & cloud

Docker · Google Cloud Run · GitHub Actions · Nginx · Vercel · Render · Cloudflare · Docker Compose · Linux (Ubuntu) · DNS & TLS · PM2 · Git & GitHub

AI & retrieval

Gemini API · RAG: BM25 + vectors (RRF) · Structured extraction · OCR (Tesseract, Poppler) · OpenAI API · Kiwi tokenizer · Chroma · hit@k · recall@k · MRR · Claude Code · Codex · Cursor

Testing & tools

Jest · Supertest · Vitest · ESLint · Prettier · pytest · Integration tests · API contract tests · PostHog · Postman

Where they run: Next.js, NestJS, GraphQL, MongoDB, Docker and Nginx in CreatorHub (live) · React, NestJS, Gemini, Neon PostgreSQL, Upstash Redis, Vercel and Render in CargoCheck AI (live) · Next.js, NestJS, pgvector, Drizzle, Gemini and Cloud Run in Sofiya (live) · Python, FastAPI, Kiwi, BM25, Chroma, Gemini and Cloud Run in Korea Visa Assistant (live beta) · FastAPI and pgvector in Lexara (collaborative).

— SELECTED PROJECT · 01 · LIVE

CreatorHub.

Solo project, 2026.07 – present: I designed, built and deployed the frontend, backend and infrastructure.

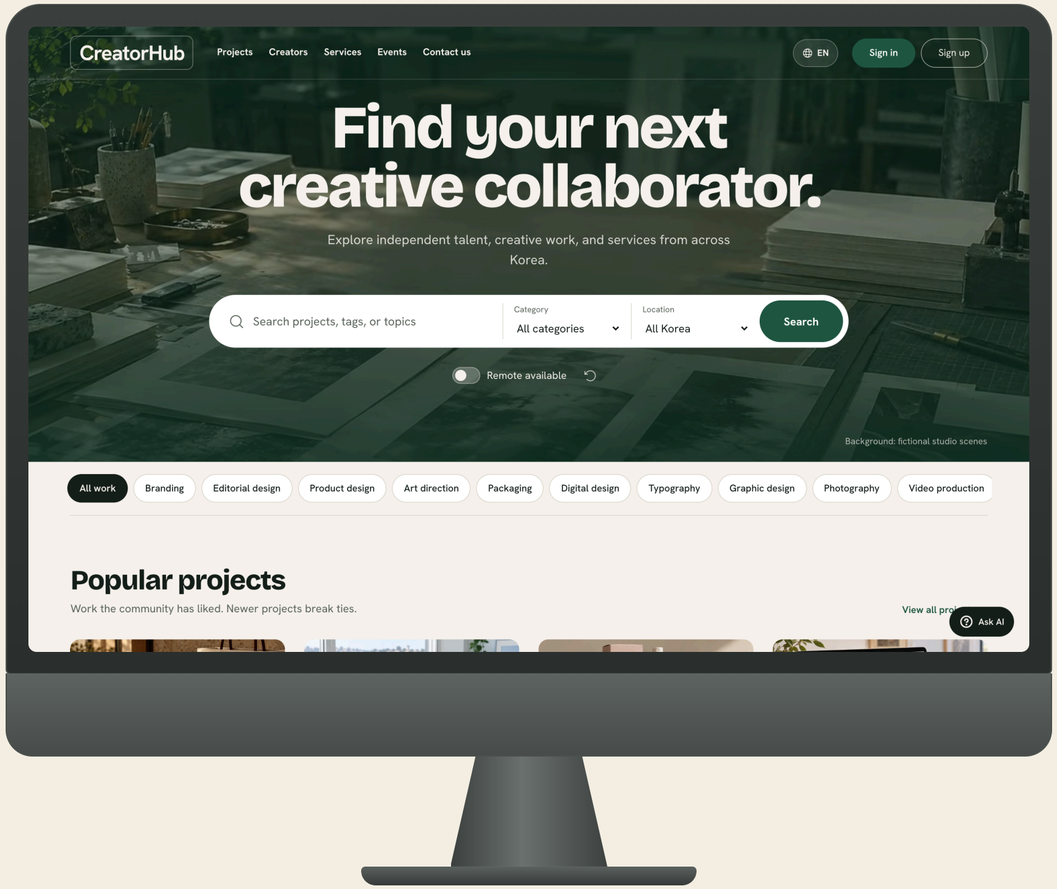
A portfolio-discovery platform where visitors find independent creators across Korea, explore their projects and services, and send private project inquiries.

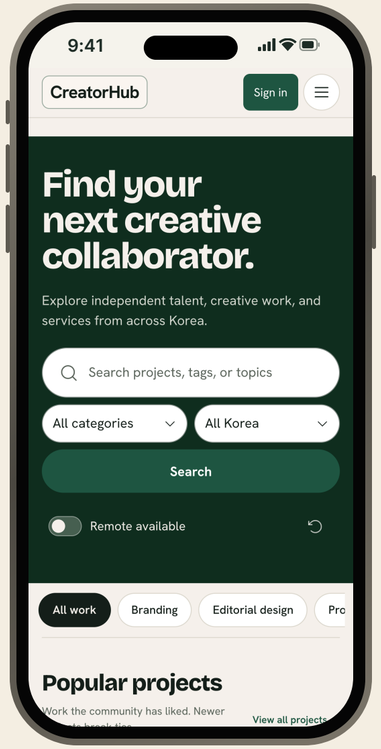
- Search by category, location and remote work
- Creator workspace with admin approval
- Private inquiries with live chat
- Gemini AI finder that answers from the site's own content

FRONTEND	Next.js · React · TypeScript
BACKEND	NestJS · GraphQL · Socket.io
DATA	MongoDB · Mongoose
INFRA	Docker · Nginx · GitHub Actions
AI	Gemini API

Visit live site ↗

Repos: CreatorHub · CreatorHub-next





— CREATORHUB · CASE STUDY

One click, one inquiry: retry-safe requests.

Problem. A double-click or a dropped connection could send the same brief or chat message twice. The server saw two new requests and would save both, with two notifications for the creator.

1
Key per form
The browser creates one key for the form and reuses it on every retry

`crypto.randomUUID()`

→

2
Check first
Inside a transaction, a known key returns the record that is already saved

`findOne(clientId, key)`

→

3
Index as a backstop
Two requests at the same moment: the unique index rejects the second insert

`partial unique index`

→

✓
One record
The losing request answers with the first record, so the client sees one result

`uq_inquiry_request`

Same key, different text
Reusing a message key for different content returns a conflict instead of silently keeping the old message.

Versions on every change
Each status change carries the version it was based on, so a stale screen cannot overwrite a newer state.

Verified
Integration tests submit the same inquiry twice and send parallel messages with one key: exactly one event, one message and one notification.

Restart-safe notification worker
Jobs are claimed with a 60-second lease and save a cursor after every batch of 100 followers. After a crash the next run resumes from the cursor; notifications are unique per follower and event.

An AI finder that cannot run up the bill
Same-site requests only, 8 messages a minute per visitor, a daily cap, retrieval that gives up after 3.5 s, and one retry on a fallback model. The Gemini key stays on the server.

The same pattern covers likes, saves, follows and comment retries.

— CREATORHUB · CASE STUDY

Testing, CI/CD and rollback.

Both repositories are tested on every push. Releases go through a separate, manually started workflow.

— WHAT IS TESTED

Backend unit · Jest
Business rules: inquiry states, passwords, rate limits

API integration · Jest
Real GraphQL on a throwaway MongoDB replica set

Frontend · Node.js
Sessions, sign-in, the AI chat route, translations

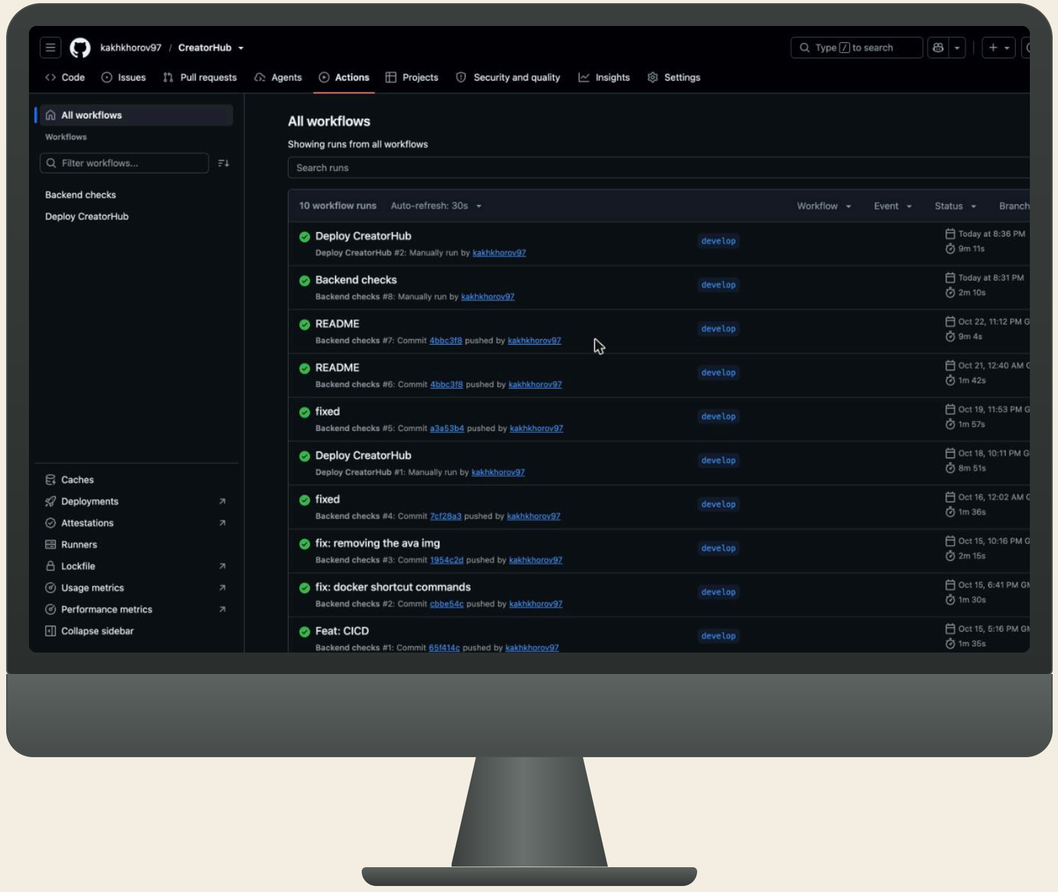
Static checks
TypeScript, ESLint, every query checked against the schema

recorded locally · 17 Oct 2026

```
$ backend   jest           ✓ 162/162 tests · 16 suites
$ backend   deploy scripts ✓ 23/23
$ frontend  node --test    ✓ 108/108
$ both      typecheck · lint · contract (119 ops) ✓ passed
= 293 automated checks recorded (162 + 23 + 108)
```

— HOW A RELEASE SHIPS

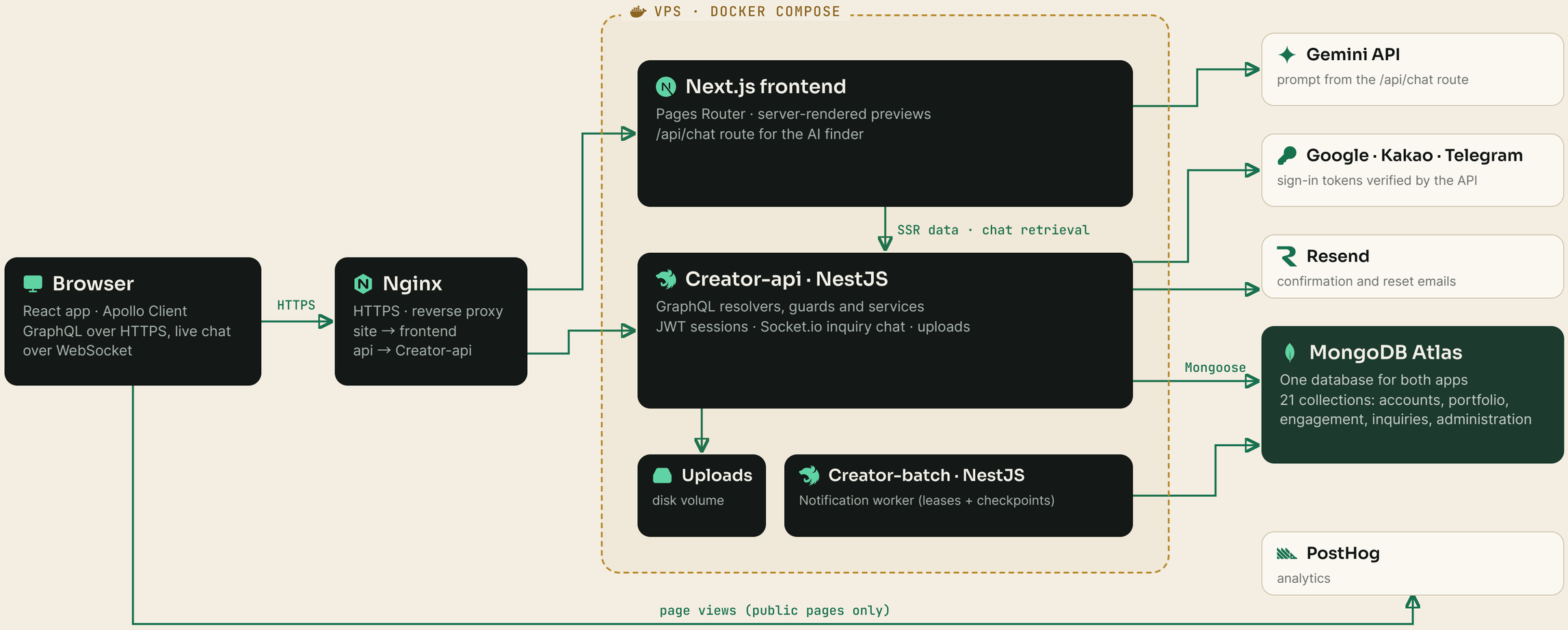
1. **Every push** to develop or main runs install, typecheck, lint, tests and build.
2. **Deploy workflow,** started by hand, re-runs all checks for one backend + frontend commit pair.
3. **Restricted SSH** to a deploy-only account that accepts nothing but “deploy” and two commit IDs.
4. **Health-checked switch:** new images get a configured window of up to 180 s to report healthy; on failure the previous images return automatically.



Screen: GitHub Actions on 24 Oct 2026, 10 of 10 runs passing, including two manual Deploy CreatorHub runs. 180 s is a configured limit, not a measured recovery time. Database writes are never rolled back.

— CREATORHUB · ARCHITECTURE

Three applications, one data model.



— SELECTED PROJECT · 02 · LIVE

Sofiya.

Live at sofiya.store. A portfolio project: real public-domain and freely licensed books, but prices, stock and orders are demo data, payments run in Toss test mode and nothing ships.

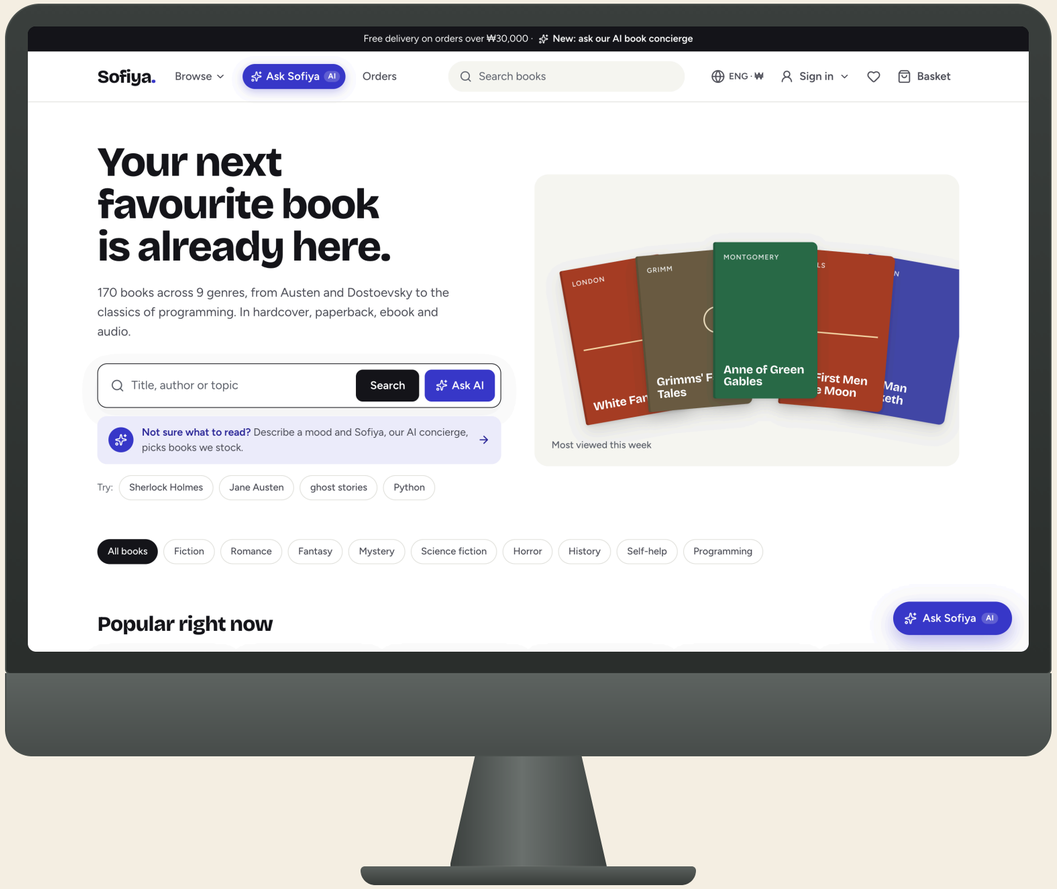
An online bookstore where readers browse and buy books, or describe the book they want in plain words and get recommendations from an AI concierge, Ask Sofiya, that cites only books in stock.

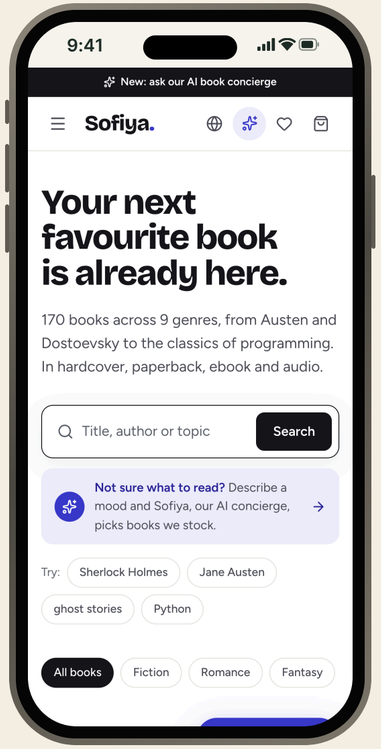
- About 170 books to browse, search and filter by genre, format and price
- Ask Sofiya: streamed answers with numbered citations
- English, Korean and Russian; won and US dollars
- Toss checkout (test mode), accounts, Google sign-in, admin with Draft with AI

FRONTEND	Next.js 15 · React 19 · Tailwind CSS
BACKEND	NestJS 11 · Drizzle ORM · Zod
DATA & AI	PostgreSQL (Neon) · pgvector · Gemini API
INFRA	Google Cloud Run · Docker · GitHub Actions

Open sofiya.store ↗

Repo: Sofiya-Bookstore (v2)





— SOFIYA · ASK SOFIYA

A book concierge that only recommends what is in stock.

1 Embed
Each book's title, author, genre and description becomes a vector
`gemini-embedding-001 · 768-d · HNSW`

→

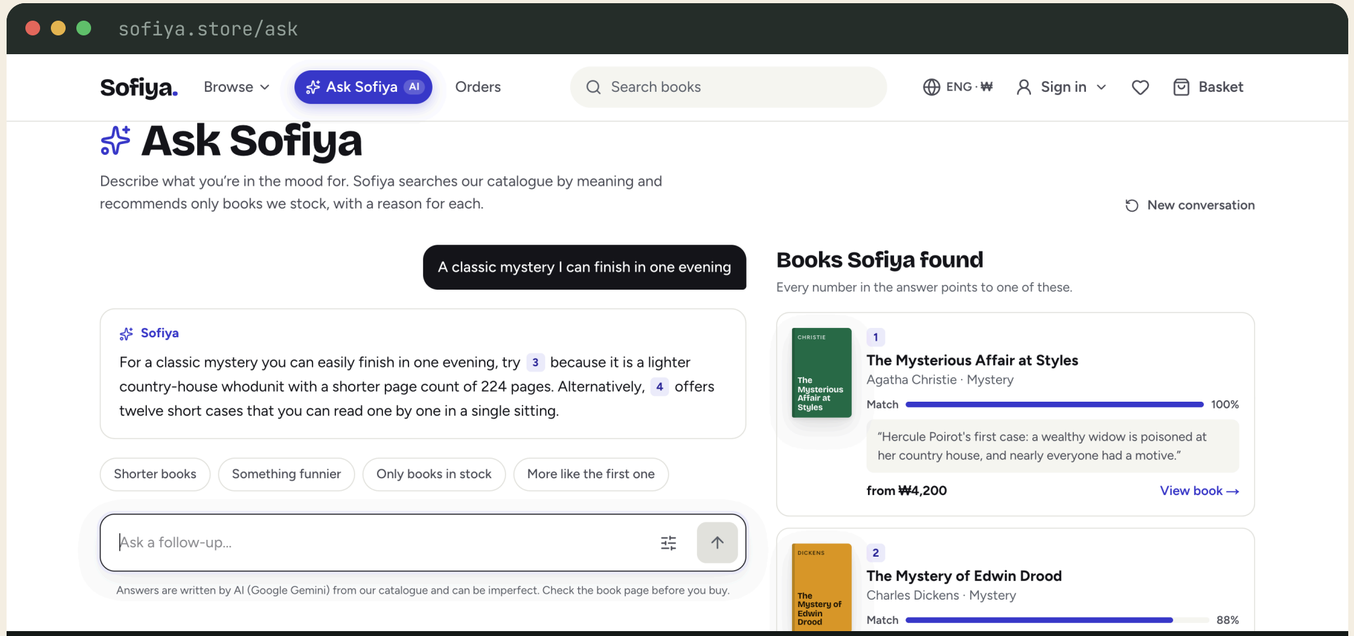
2 Retrieve twice
Vector similarity in pgvector and Postgres full-text search; off-topic questions fall below a 0.60 floor
`pgvector + full-text`

→

3 Fuse
Reciprocal Rank Fusion merges both rankings; the top five go to the model
`RRF · top 5`

→

✓ Answer and verify
Gemini streams the answer; the server drops any citation that is not one of the five
`fallback model on failure`



Live answer, 28 Oct 2026: "A classic mystery I can finish in one evening". Numbered citations point to the books on the right.

— MEASURED RETRIEVAL · 45 FIXED QUESTIONS

	hit@1	hit@5	MRR
Hybrid (site)	93%	100%	0.95
Keyword only	75%	85%	0.79
Meaning only	100%	100%	1.00

40 answerable questions (direct, paraphrased, themed) and 5 unanswerable ones; every unanswerable question returned no books. I wrote the questions with the catalogue in view, so the scores are optimistic and serve as a regression check. Hybrid stays because keyword search still works when Gemini is slow or down.

— SOFIYA · ENGINEERING

Payments, security and deployment.

Order saved
Stock reserved; an idempotency key stops double orders

→

Toss window
Card, Toss Pay, KakaoPay or NaverPay, in test mode

→

Server confirms
With Toss, for the amount it stored, never one from the browser

→

Paid or expired
Unpaid orders expire and return their stock

→

Cancel = refund
Cancelling a paid order refunds it automatically

Security

Google OAuth with state and PKCE, hashed one-time email and reset tokens, sessions ended on password change, per-visitor rate limits, HSTS and frame-ancestors headers, uploads re-encoded to WebP to strip EXIF/GPS.

Resilience

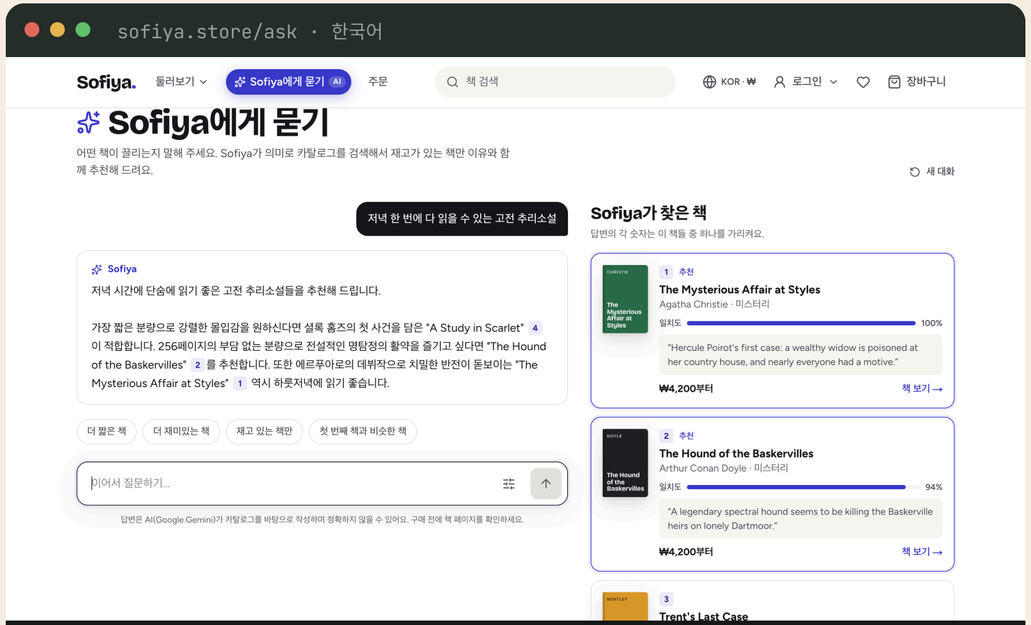
A slow or failing model falls back to a second Gemini model, and the matching books still show if both fail. Rate limits fall back to memory if Redis is down. Emails send in the background.

Deployment

Separate web and API services on Google Cloud Run in Seoul, Secret Manager and Artifact Registry, keyless GitHub Actions deploys with Workload Identity Federation; migrations run before the new API takes traffic.

Tests

About 260 automated tests: ~120 API unit, ~115 API end-to-end against a real Postgres (Jest + Supertest), ~30 frontend and shared (Vitest).



The same question in Korean: the interface and the AI answer switch language. Test-mode payments; no real money is charged.

— SELECTED PROJECT · 03 · LIVE BETA

Korea Visa Assistant

Live public beta on Google Cloud Run. General information from selected official sources: it does not decide eligibility or give legal advice.

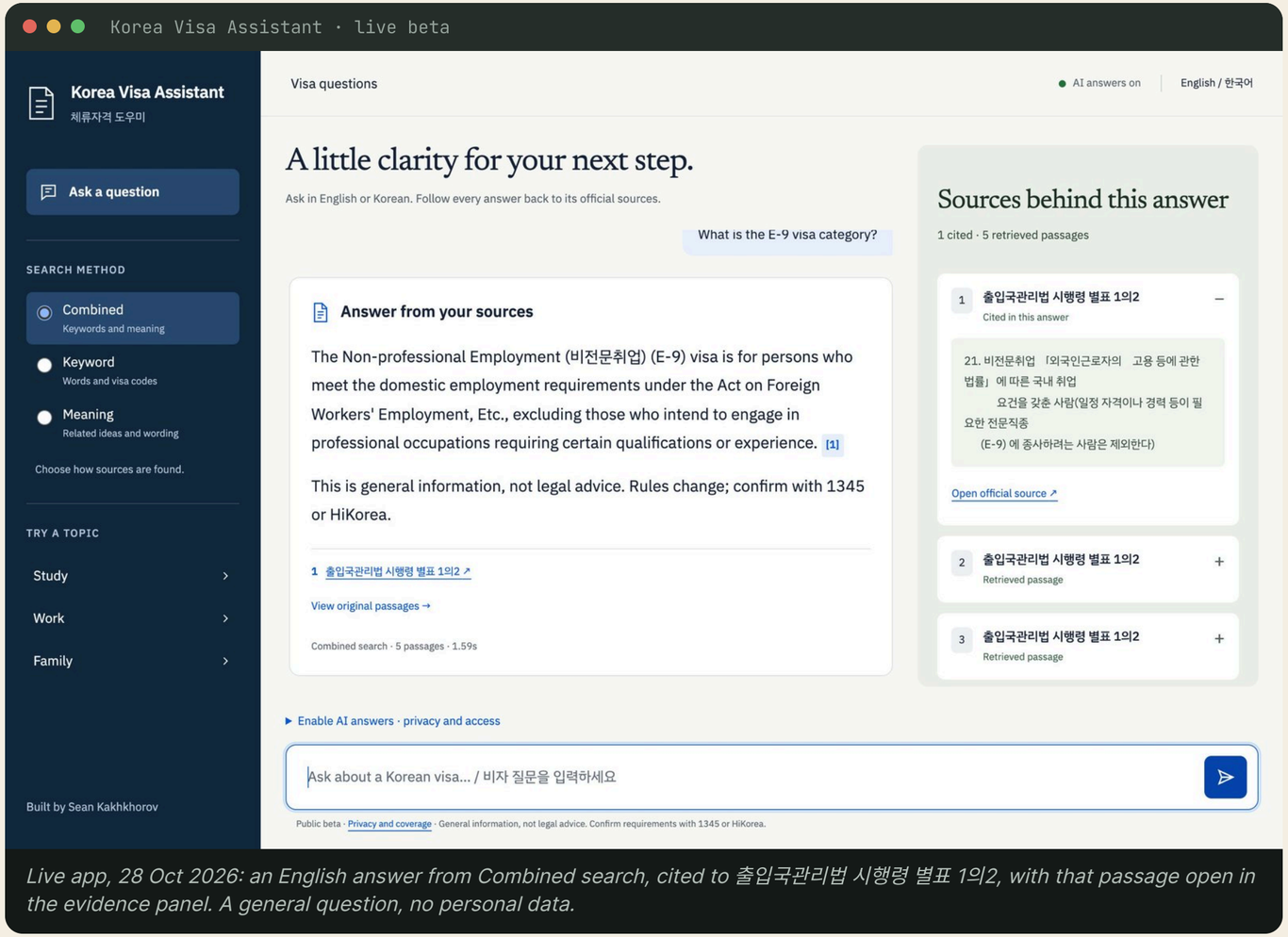
Search selected official Korean visa sources and get source-grounded explanations in English or Korean. Keyword search is free by default; AI answers need an 18+, South Korea and privacy acknowledgment.

- Keyword, meaning and combined search
- Gemini claims checked against source quotes
- Citations open the official passage
- Abstains, or falls back to original passages

BACKEND	Python · FastAPI · Pydantic
RETRIEVAL	Kiwi · BM25 · Chroma
AI	Gemini API
INFRA	Cloud Run · Cloud Storage · Docker
TESTS	pytest · GitHub Actions

Open the live app ↗

Repo: korea-visa-rag



Live app, 28 Oct 2026: an English answer from Combined search, cited to 출입국관리법 시행령 별표 1의2, with that passage open in the evidence panel. A general question, no personal data.

Public corpus: 44 selected statute chunks covering 21 broad visa categories, not a complete immigration database. HiKorea guide extracts stay in the repository and are not in the live corpus.

— KOREA VISA ASSISTANT · CASE STUDY

Retrieval built for Korean legal text.

Problem. Korean law is structured as article › paragraph › item, and questions use exact codes like D-10 or E-7-1. Fixed-size chunks and generic tokenizers split both apart, so the right article is missed.

1

Chunk by article

Statutes split per 제N조 or 제N조의M, keeping paragraphs and items; guides split by section, then windowed

`article label + source URL`

→

2

Tokenize

Kiwi morphological analysis; visa codes kept whole so a code search matches exactly

`D-10 · E-7-1`

→

3

Retrieve twice

BM25 for exact terms and codes, Chroma with Gemini embeddings for meaning

`gemini-embedding-001`

→

✓

Fuse

Reciprocal Rank Fusion merges both rankings into one top five

`RRF · k = 60`

Why keep codes whole

Split into pieces, “E-7-1” would also match E-7 and every other code. One token keeps a code search exact.

Why hybrid

Keywords catch codes, meaning catches paraphrases. On 32 labelled questions, fusion lifted MRR from 0.702 (BM25) and 0.827 (vector) to 0.935.

Every claim points to a passage

Gemini returns claims with quotes; the app checks each quote against its source and cites only checked claims. No support: it abstains. A failed check: it shows the original passages.

Python · FastAPI · Pydantic · Uvicorn · kiwipiepy · rank-bm25 · Chroma · google-genai · python-telegram-bot

— KOREA VISA ASSISTANT · TESTING AND LIMITS

What is measured, and what is not.

— SAVED RETRIEVAL EVALUATION · 26 OCT 2026 · 32 LABELLED QUESTIONS

	hit@5	recall@5	MRR
BM25	0.812	0.703	0.702
Vector	0.969	0.891	0.827
Hybrid (RRF)	0.969	0.891	0.935

Synthetic, source-labelled questions over the 44-chunk corpus; not independently annotated. This measures retrieval, not legal-answer accuracy. The 36-question AI run (29 responses, 7 failures, 10 abstentions) predates the current fallback and was not rerun.

Checked on 2d29f46, 28 Oct 2026	Result
Python tests (pytest)	125 / 125 pass
JavaScript helper tests (node:test)	5 / 5 pass
Live /api/health	44 chunks · 3 modes
8 live AI questions	3 cited · 1 abstained · 3 fallback · 1 rejected

The JavaScript tests cover rendering and navigation helpers, not a browser suite. The live questions are an observation from this capture, not a measurement.

— KNOWN LIMITS · NEXT STEPS

Claim support
A matching quote does not prove the claim follows. Next: a reviewed claim-level test set.

Independent questions
Earlier turns are not sent, so each question stands alone.

Request recovery
No timeout, cancel or retry; a failed meaning search does not fall back to keyword search.

Coverage
No full document checklists, many subtypes, student-work rules or individual eligibility.

Source dates and labels
Collection dates are stored but not shown; annex passages share one title and link.

Korean interface
The fallback notice and many messages are still English.

Telegram adapter
Lacks the web's identifier, acknowledgment and quota checks and logs raw questions on errors.

— HOW THE BETA RUNS

Snapshots
A pinned index snapshot from Cloud Storage, checked by SHA-256; no embedding calls at startup.

Privacy
Emails, phone, passport and registration numbers are rejected before any AI call.

Shared limits
Request, daily AI and concurrency limits in one Cloud Storage ledger shared by every instance.

Releases
Manual GitHub Actions deploy through Workload Identity Federation, with rollback and a smoke test.

— SELECTED PROJECT · 04 · LIVE

CargoCheck AI.

Live, with Gemini extraction in production. An early build: no customs certification and no automatic approval.

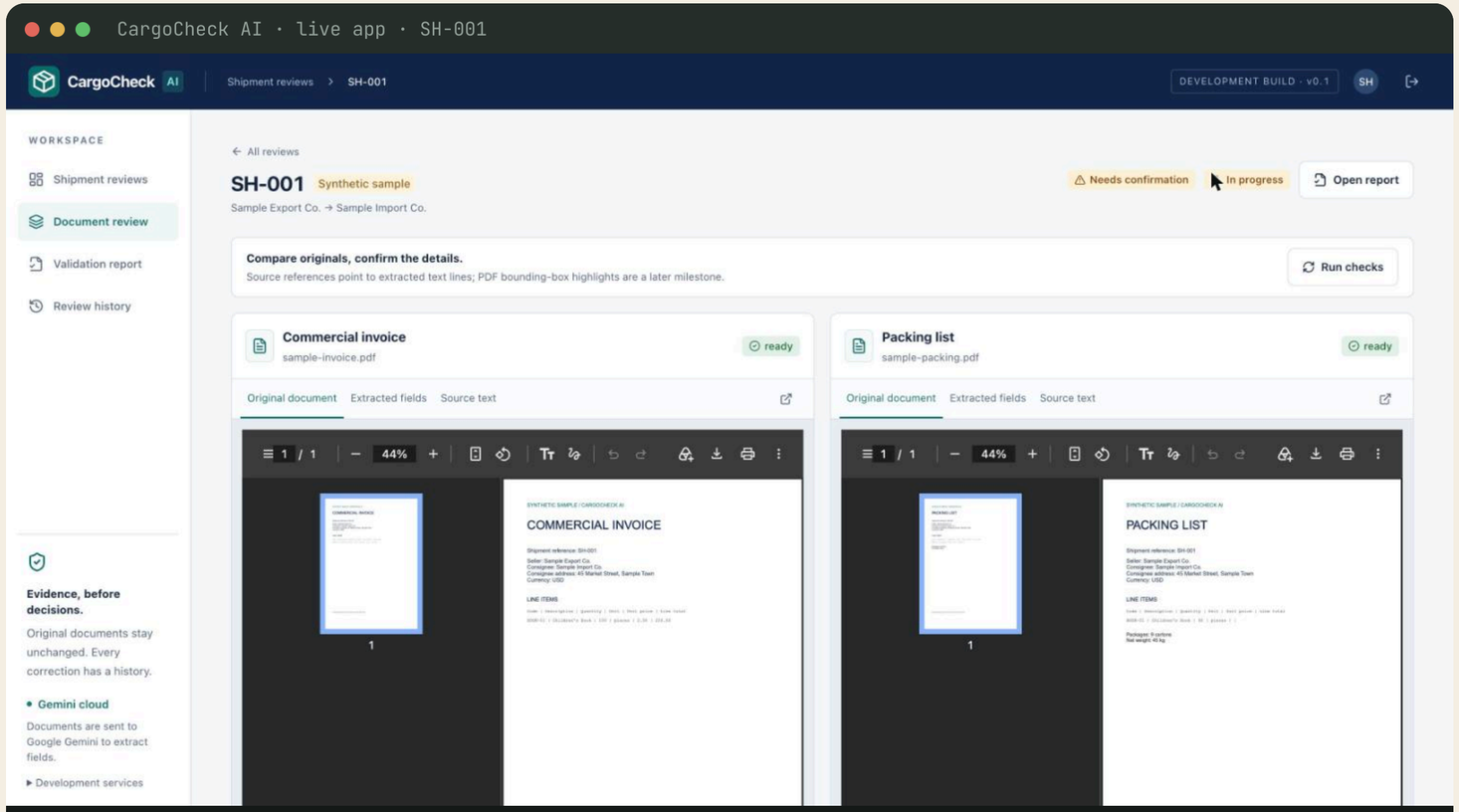
A review app for invoices and packing lists: Gemini extracts the fields, a reviewer confirms each value, rules flag mismatches, and every decision is recorded. My International Trade degree taught me how these documents are checked; CargoCheck AI is that workflow, built as software.

- Upload both documents; Gemini extracts the fields in a background job
- Rule checks, one decision per flagged item, history and a PDF report

FRONTEND	React · TypeScript · Vite
BACKEND	NestJS · Zod · decimal.js
AI	Gemini API · Tesseract · Poppler
DATA	Neon · Upstash · BullMQ
INFRA	Vercel · Render · Docker

Open the live app ↗

Repo: cargocheck-ai



Live app, 28 Oct 2026: both original documents processed in Gemini cloud mode (synthetic sample).

Known issues, listed openly: extracted source lines can be off, confirm-all can skip an unsaved edit, uploads sit on the container disk, no password recovery yet, CI runs Node tests only, and the mobile view hides the Gemini notice.

— CARGOCHECK AI · EXTRACTION

From upload to reviewable fields.

Problem. Reviewers retype and cross-check invoice and packing-list values by hand. A model can misread a value, so nothing it extracts counts until a person confirms it against the original.

1 — Upload
Invoice and packing list; file signatures checked; originals never rewritten

`PDF · JPG · PNG · 10 MB`

→

2 — Extract
A BullMQ job sends both documents to Gemini structured output

`gemini-3.5-flash-lite`

→

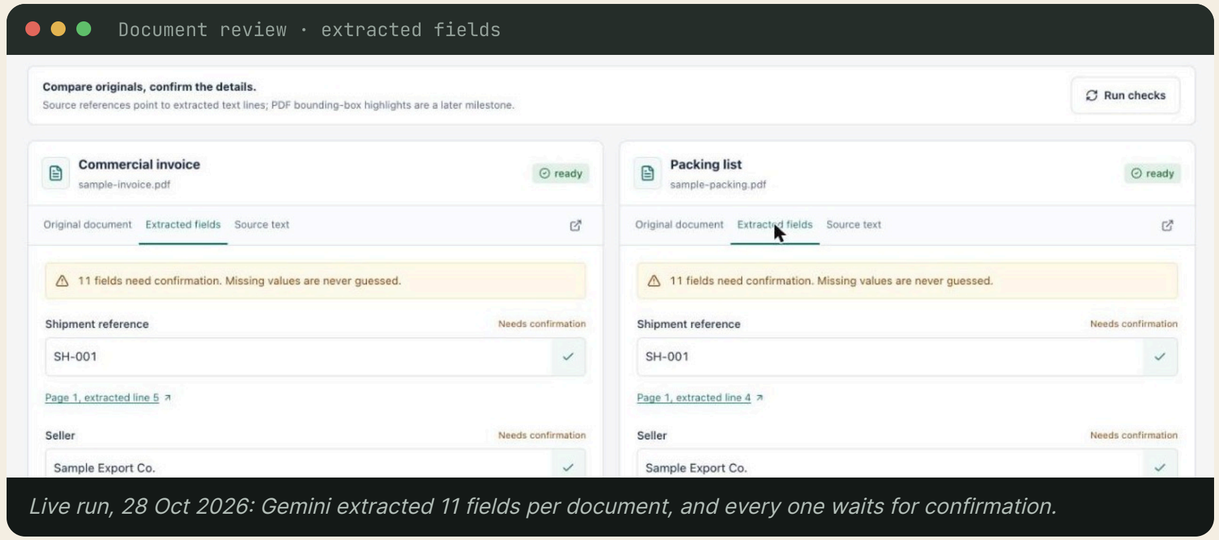
3 — Validate
The response is parsed against a Zod schema; unknown values stay empty

`missing values stay null`

→

✓ — Confirm
The reviewer confirms or corrects each value; the original value is kept

`needs confirmation → confirmed`



Live run, 28 Oct 2026: Gemini extracted 11 fields per document, and every one waits for confirmation.

Never guessed

A value the model cannot find stays empty. The banner says so: missing values are never guessed.

Two modes

Production uses Gemini. A local mode runs English Tesseract OCR and a labelled-template parser with no AI key.

Line links are a hint

Each value links to the page and line the model reported. Those line numbers can be off, so they are not proof (a known issue).

Deterministic rules, human decisions.

Once both documents are confirmed, rules compare the confirmed values. A rule only flags a difference; the reviewer decides what it means and writes down why.

— WHAT IS COMPARED

- Shipment reference, seller, consignee and address
- Explicit product codes; ambiguous matches stay review cases
- Quantities in compatible units; incompatible units are flagged, never converted
- Line arithmetic in decimal.js: quantity × unit price = stated line total
- Net weight is shown as not compared in this release

One decision per flag

The review cannot be completed until every flagged check has a recorded decision.

Not an approval

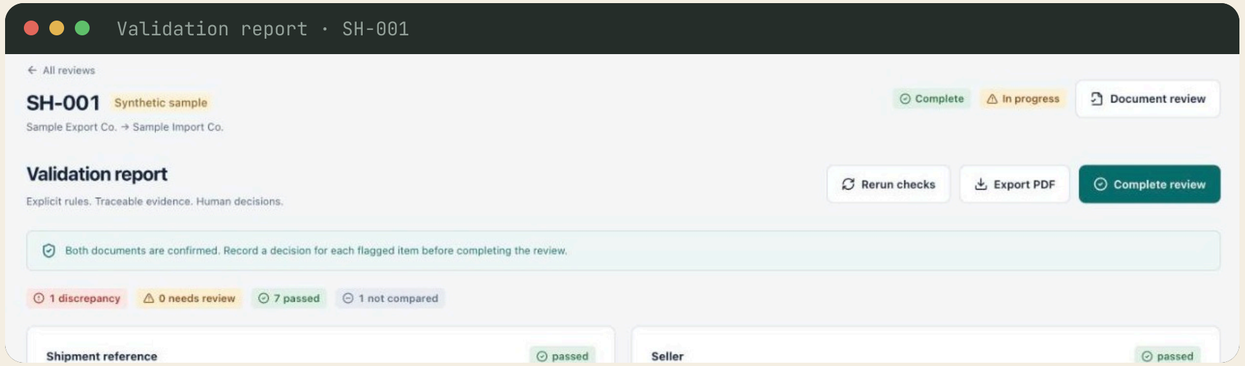
Completing a review records the reviewer's work. It does not approve a shipment.

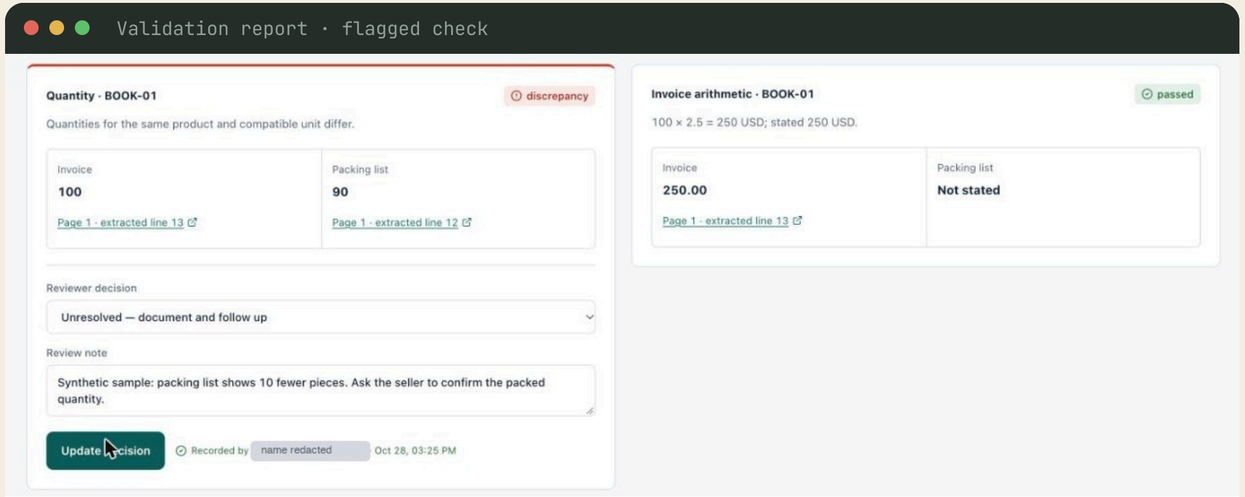
Corrections reopen the review

The original value is kept, earlier decisions are cleared, and an edit from an older copy gets 409.

Accounts and tests

Scrypt password hashes, hashed session tokens, ownership and upload checks; 42/42 tests in CI.





Live, 28 Oct 2026: 100 invoiced vs 90 packed pieces, with the recorded decision. Synthetic values; the reviewer's name is redacted.

— SELECTED PROJECT · 05 · LIVE

Lexara.

Live, collaborative project; I am a contributor. I worked on the FastAPI backend: API endpoints, upload processing, database models and Alembic migrations. Our code was pushed through my collaborator's GitHub account; my repository is a fork of MakhmudSD/Lexara.

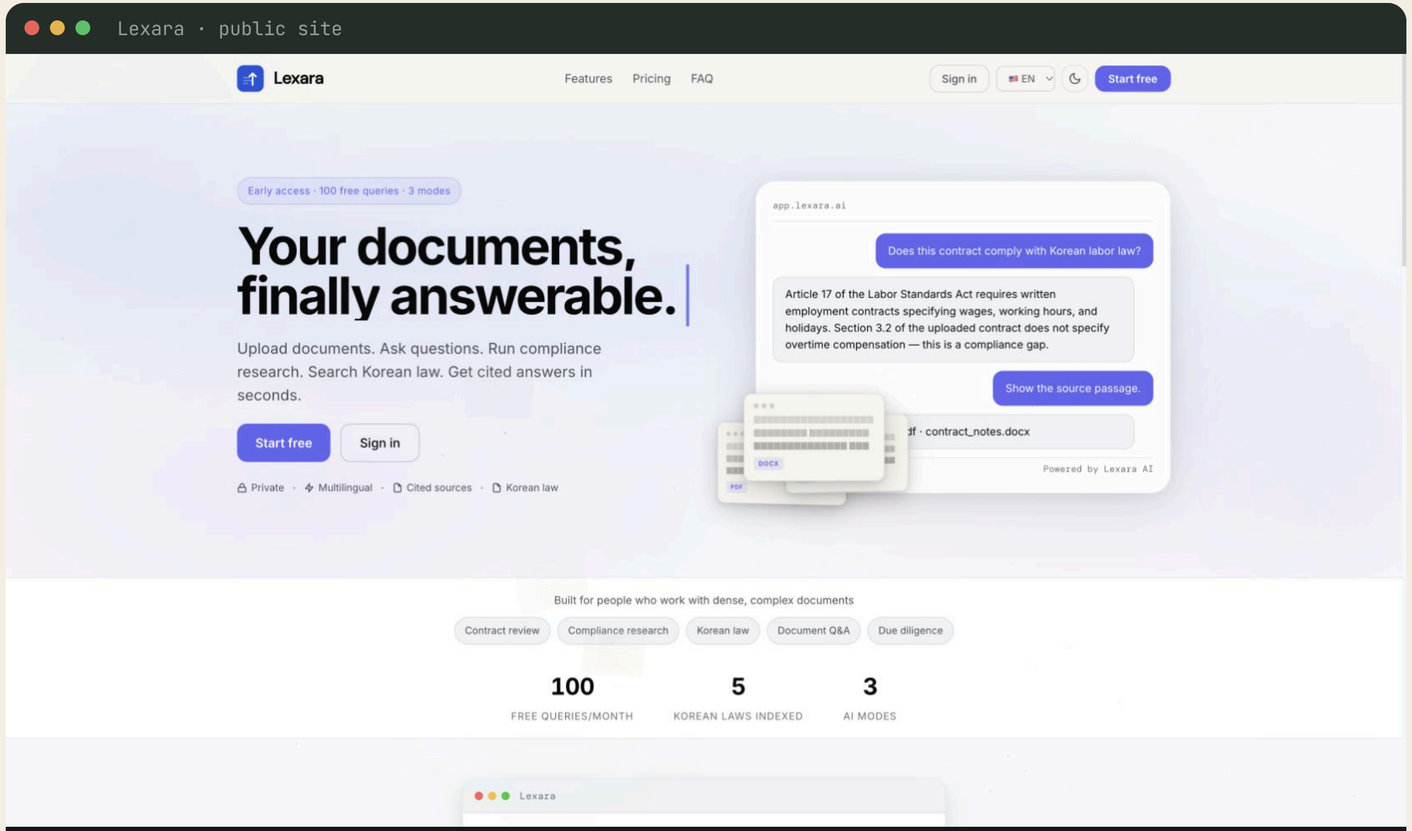
A document Q&A platform: uploaded documents are organised into workspaces, and answers cite the passages they come from.

— TEAM ARCHITECTURE · MY PART: BACKEND

- FastAPI with PostgreSQL, SQLAlchemy and Alembic
- pgvector similarity plus BM25, merged with RRF
- Uploads return 202 and are processed as background tasks, with status polling
- React/Vite frontend; OpenAI models; optional R2 storage

Public site ↗

Repo: **kakhkhorov97/Lexara** (fork of **MakhmudSD/Lexara**)



Lexara public landing page, 22 Oct 2026. The answers in the embedded preview are demonstration content.

The older README describes FAISS; the inspected source uses pgvector. Sign-in, upload and chat were not tested for this portfolio.

— CONTACT

From idea to production

EMAIL	kakhkhorov97@gmail.com
PHONE	010-7901-7555
KAKAOTALK	kakhkhorov_A
GITHUB	github.com/kakhkhorov97
PORTFOLIO	sean-kakhkhorov.site
CREATORHUB	creatorhub.site
SOFIYA	sofiya.store
KOREA VISA ASSISTANT	Open the live beta ↗
CARGOCHECK AI	Open the live app ↗
ADDRESS	Suwon, Gyeonggi-do, South Korea



Portfolio
sean-kakhkhorov.site

CreatorHub
creatorhub.site

Sofiya
sofiya.store